

Códigos de Status HTTP e Tratamento de Exceções

Entenda como o servidor se comunica com o cliente e como usar **try...catch** em JavaScript/Node.js para tratar cada tipo de resposta de forma profissional e segura.

DESENVOLVIMENTO DE SOFTWARE

JAVASCRIPT / NODE.JS



Agenda

01

Introdução à Comunicação HTTP

O que são códigos de status e anatomia de uma resposta

02

As Famílias de Códigos

Visão geral das faixas 2xx, 3xx, 4xx e 5xx

03

Sucesso (2xx) e o Bloco Try

Como o fluxo normal continua após uma resposta bem-sucedida

04

Erros do Cliente (4xx)

Alguns dos erros mais comuns gerados pelo lado do cliente.

05

Erros do Servidor (5xx)

Alguns dos erros mais comuns gerados no lado do servidor.

06

Resumo

Mapa completo para tratar respostas HTTP de forma profissional em JavaScript/Node.js.

CAPÍTULO 1

Introdução à Comunicação HTTP

Toda conversa entre cliente e servidor segue um protocolo. Entender esse diálogo é o primeiro passo para construir aplicações robustas.



O Que São Códigos de Status HTTP?

Quando um cliente (navegador, app, script) faz uma requisição a um servidor, o servidor sempre responde com um **código de status HTTP** — um número de 3 dígitos que resume o resultado da operação.

Requisição do Cliente

O cliente pede um recurso: uma página, dados de uma API, ou envia um formulário.

Resposta do Servidor

O servidor processa e responde com um **código de status** indicando sucesso, redirecionamento ou erro.

 Pense no código de status como o "resultado" de uma operação: ele é a primeira coisa que você deve checar ao lidar com respostas HTTP.

Anatomia de uma Resposta HTTP

Toda resposta HTTP é composta por três partes fundamentais. Conhecê-las é essencial para interpretar e tratar respostas corretamente.

1

Status Code

Número de 3 dígitos (ex: 200, 404, 500) que indica o resultado. É a informação mais crítica para o tratamento de erros.

2

Headers

Metadados da resposta: tipo do conteúdo (Content-Type), tokens de autenticação, cache, CORS, entre outros.

3

Body

O conteúdo da resposta em si — geralmente JSON em APIs REST. Pode ser vazio em respostas 204 No Content.

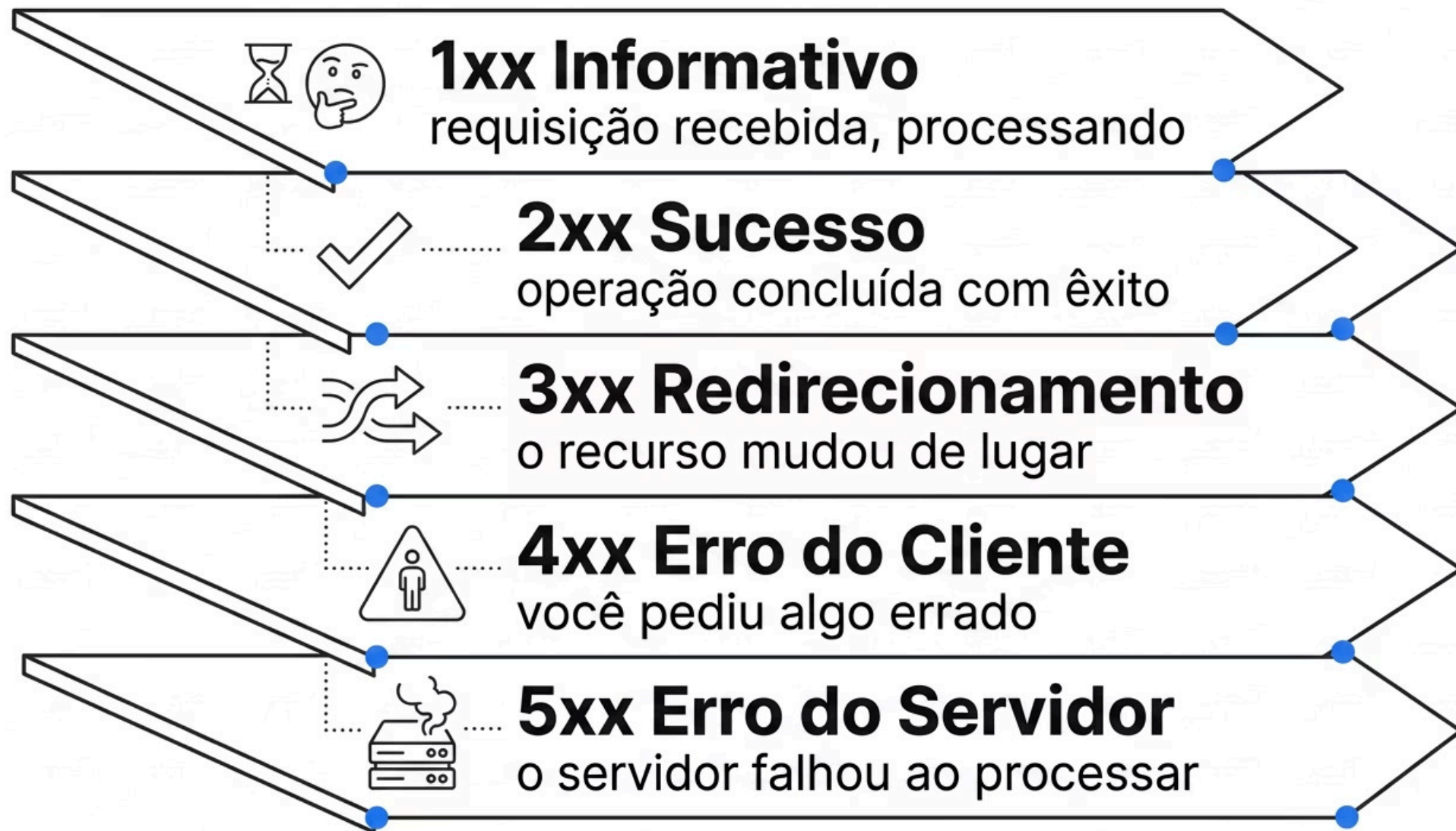
HTTP/1.1 200 OK

Content-Type: application/json

```
{ "id": 1, "nome": "Ana" }
```

As Famílias de Códigos HTTP

Os códigos de status são agrupados em **cinco famílias**, definidas pelo primeiro dígito. Cada família representa uma categoria de resultado completamente diferente.



Para desenvolvimento de APIs, as famílias mais relevantes são **2xx**, **4xx** e **5xx** — e é nelas que focaremos.

Visão Geral das Famílias

2xx — Sucesso

A operação foi recebida, entendida e processada com êxito. O fluxo normal continua.

3xx — Redirecionamento

O recurso mudou de lugar. O cliente deve seguir o novo endereço indicado no header `Location`.

4xx — Erro do Cliente

O cliente enviou uma requisição inválida. O problema está do lado de quem fez o pedido.

5xx — Erro do Servidor

O servidor falhou ao processar uma requisição válida. O problema está na infraestrutura ou no código do back-end.

Códigos de Sucesso (2xx) e o Bloco Try

Quando o servidor retorna um código 2xx, significa que tudo correu bem. No contexto do `try...catch`, o bloco **try** executa sem desvios.

200 OK

O código mais comum. A requisição foi bem-sucedida e o body contém o resultado esperado. Usado em `GET`, `PUT` e `DELETE`.

201 Created

Um novo recurso foi criado com sucesso. Geralmente resposta de um `POST`. O header `Location` pode apontar para o novo recurso.

204 No Content

Operação bem-sucedida, mas sem body na resposta. Comum em `DELETE`.

Fluxo no Try

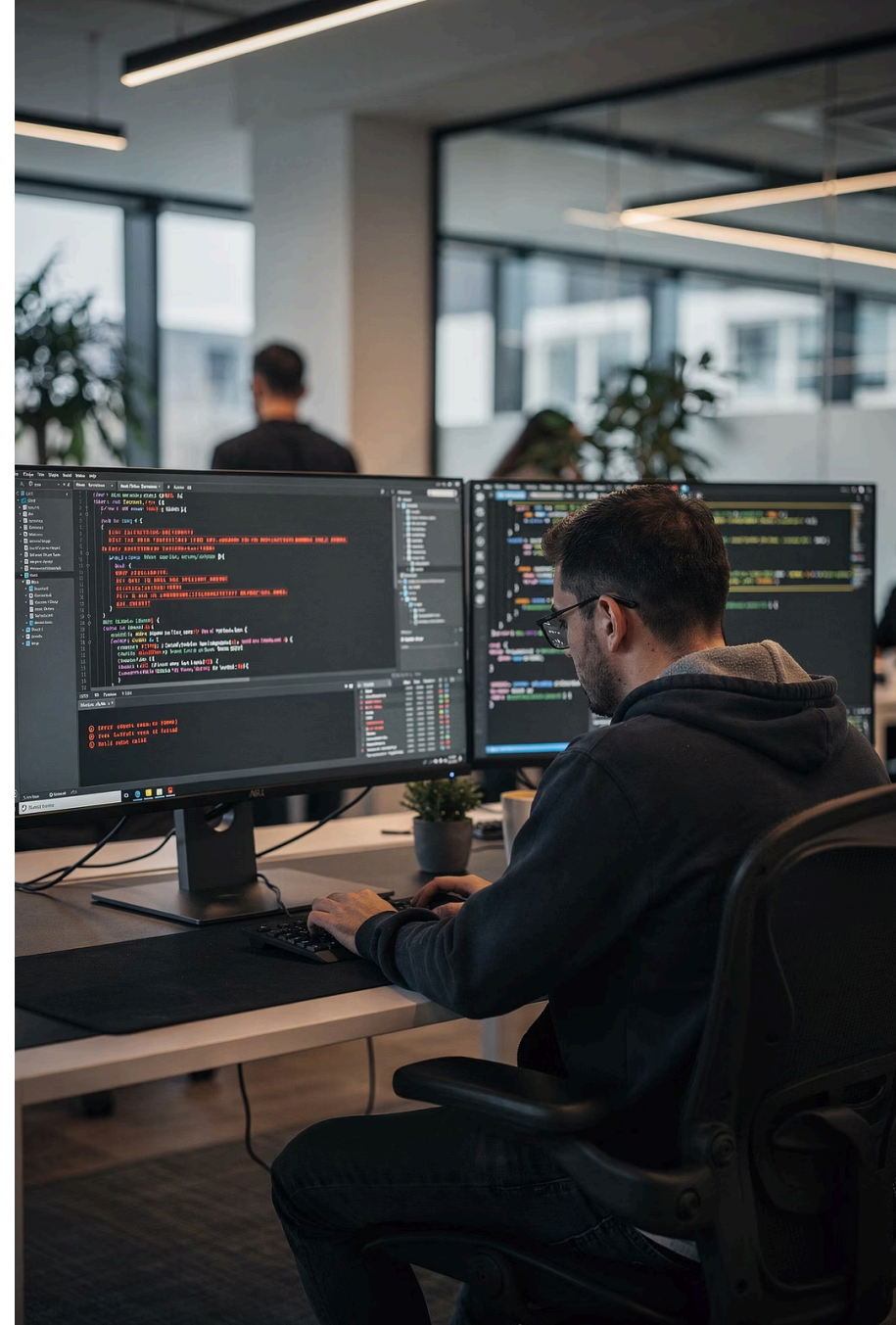
Com um 2xx, o código dentro do bloco `try` continua sua execução de forma linear, sem lançar erros:

```
try {  
  const res = await fetch('/api/users');  
  const data = await res.json();  
  //
```

CAPÍTULO 4

Erros do Cliente (4xx)

Os erros 4xx indicam que **o problema está na requisição enviada pelo cliente**. São erros previsíveis e devem ser tratados de forma proativa, com validações claras.



Os Principais Códigos 4xx

Código	Nome	Quando ocorre?	Ação recomendada
400	Bad Request	Dados inválidos, JSON malformado, campo obrigatório ausente	Validar os dados antes de enviar; exibir mensagem ao usuário
401	Unauthorized	Token ausente ou expirado	Redirecionar para login; renovar token
403	Forbidden	Token válido, mas sem permissão para o recurso	Exibir erro de permissão; não tentar novamente
404	Not Found	Recurso ou endpoint inexistente	Verificar a URL; exibir "não encontrado" ao usuário

 Erros 4xx **não devem ser logados como críticos** no servidor — eles são responsabilidade do cliente e fazem parte do fluxo normal da API.

Tratando Erros 4xx com throw new Error

A Fetch API do JavaScript **não lança exceções automaticamente** para status 4xx. Você precisa verificar o status e lançar o erro manualmente para que o `catch` seja ativado.

```
async function buscarUsuario(id) {  
  try {  
    const res = await fetch(`/api/users/${id}`);  
  
    if (res.status === 404) {  
      throw new Error('404: Usuário não encontrado.');    }  
  
    if (res.status === 401) {  
      throw new Error('401: Sessão expirada. Faça login novamente.');    }  
  
    if (res.status === 403) {  
      throw new Error('403: Você não tem permissão para acessar este recurso.');    }  
  
    const usuario = await res.json();  
    return usuario; //
```

Erros do Servidor (5xx)

Erros 5xx são os mais críticos: indicam que **o servidor falhou** ao processar uma requisição válida. São imprevisíveis e o bloco `catch` é nossa principal defesa.

500 Internal Server Error

O código mais genérico. O servidor encontrou uma condição inesperada — pode ser um bug, uma exception não tratada, ou uma falha de banco de dados.

Exige análise de logs imediata.

502 Bad Gateway

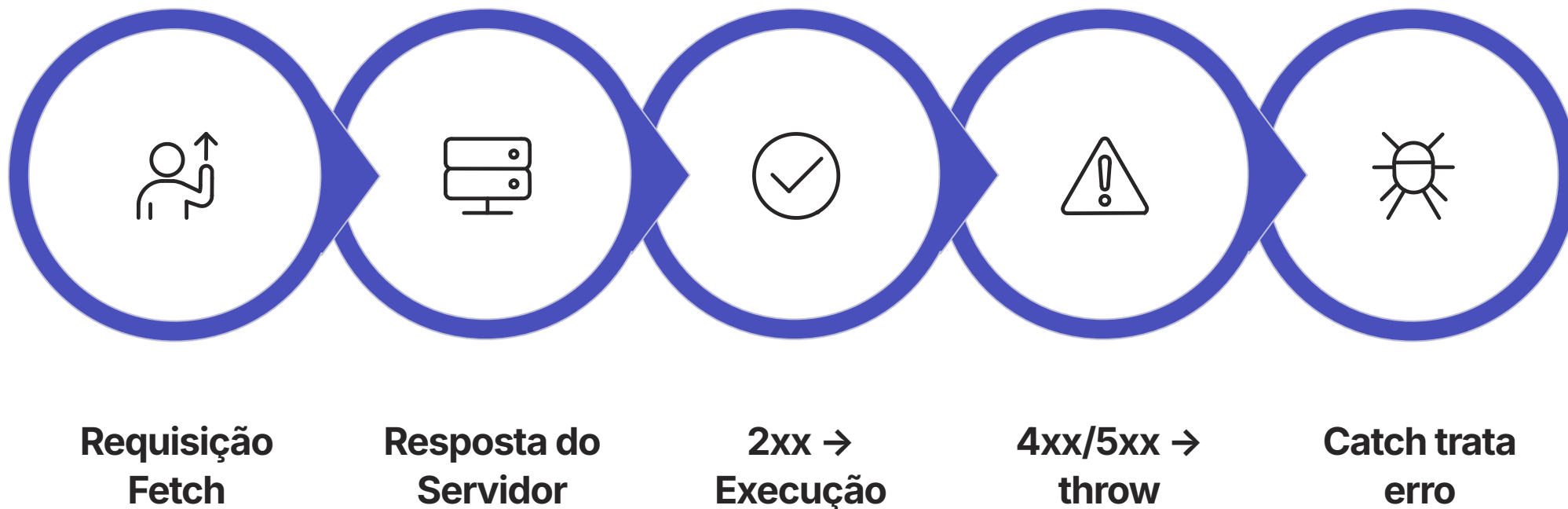
O servidor estava atuando como proxy e recebeu uma resposta inválida do servidor upstream. Comum em arquiteturas de microsserviços.

503 Service Unavailable

O servidor está temporariamente fora do ar — sobrecarga, reinicialização ou manutenção. Pode ser resolvido com retry automático após alguns segundos.

Visualizando o Fluxo Completo

Veja como o controle de fluxo se comporta para cada família de código de status dentro do bloco `try...catch`:



A chave está em transformar respostas HTTP em exceções JavaScript intencionalmente, usando `throw`, para que o fluxo sempre chegue ao `catch` quando algo der errado.

Resumo

Você agora tem o mapa completo para tratar respostas HTTP de forma profissional em JavaScript/Node.js.

2xx no Try

Fluxo normal. Execute sua lógica com segurança e processe os dados retornados.

4xx com throw

Verifique `res.ok` ou `res.status` e lance erros específicos para cada código.

5xx no Catch

Capture a exceção, logue internamente com contexto e retorne uma resposta genérica ao cliente.

Segurança Sempre

Nunca exponha stack traces. Diferencie logs por severidade. Use ferramentas de monitoramento em produção.

- ✔ Dominar esses padrões significa construir APIs mais resilientes, seguras e fáceis de manter — qualidades valorizadas em qualquer equipe de desenvolvimento profissional.